5.6 Self-attention and positional encoding

- in deep learning, we often use CNNs or RNNs to encode a sequence
- now, with attention mechanisms, imagine that we feed a sequence of tokens into attention pooling, so that the same set of tokens act as queries, keys, and values
- specifically, each query attends to all the key-value pairs and generates one attention output
- since the queries, keys, and values come from the same place, this performs *self-attention*, which is also called *intra-attention*
- in this section, we will discuss sequence encoding using self-attention, including using additional information for the sequence order

5.6 Self-attention and positional encoding

- given a sequence of input tokens $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n$, where any $\boldsymbol{x}_i \in \mathbb{R}^d$ $(1 \leq i \leq n)$, its self-attention outputs a sequence of the same length $\boldsymbol{y}_1, \ldots, \boldsymbol{y}_n$, where:

$$\boldsymbol{y}_i = f(\boldsymbol{x}_i, (\boldsymbol{x}_1, \boldsymbol{x}_1), \ldots, (\boldsymbol{x}_n, \boldsymbol{x}_n)) \in \mathbb{R}^d,$$

  according to the definition of attention pooling f in (5)
- let us compare architectures for mapping a sequence of n tokens to another sequence of equal length, where each input or output token is represented by a d-dimensional vector
- specifically, we will consider CNNs, RNNs, and self-attention
- we will compare their computational complexity, sequential operations, and maximum path lengths
- note that sequential operations prevent parallel computation, while a shorter path between any combination of sequence positions makes it easier to learn long-range dependencies within the sequence

5.6 Self-attention and positional encoding

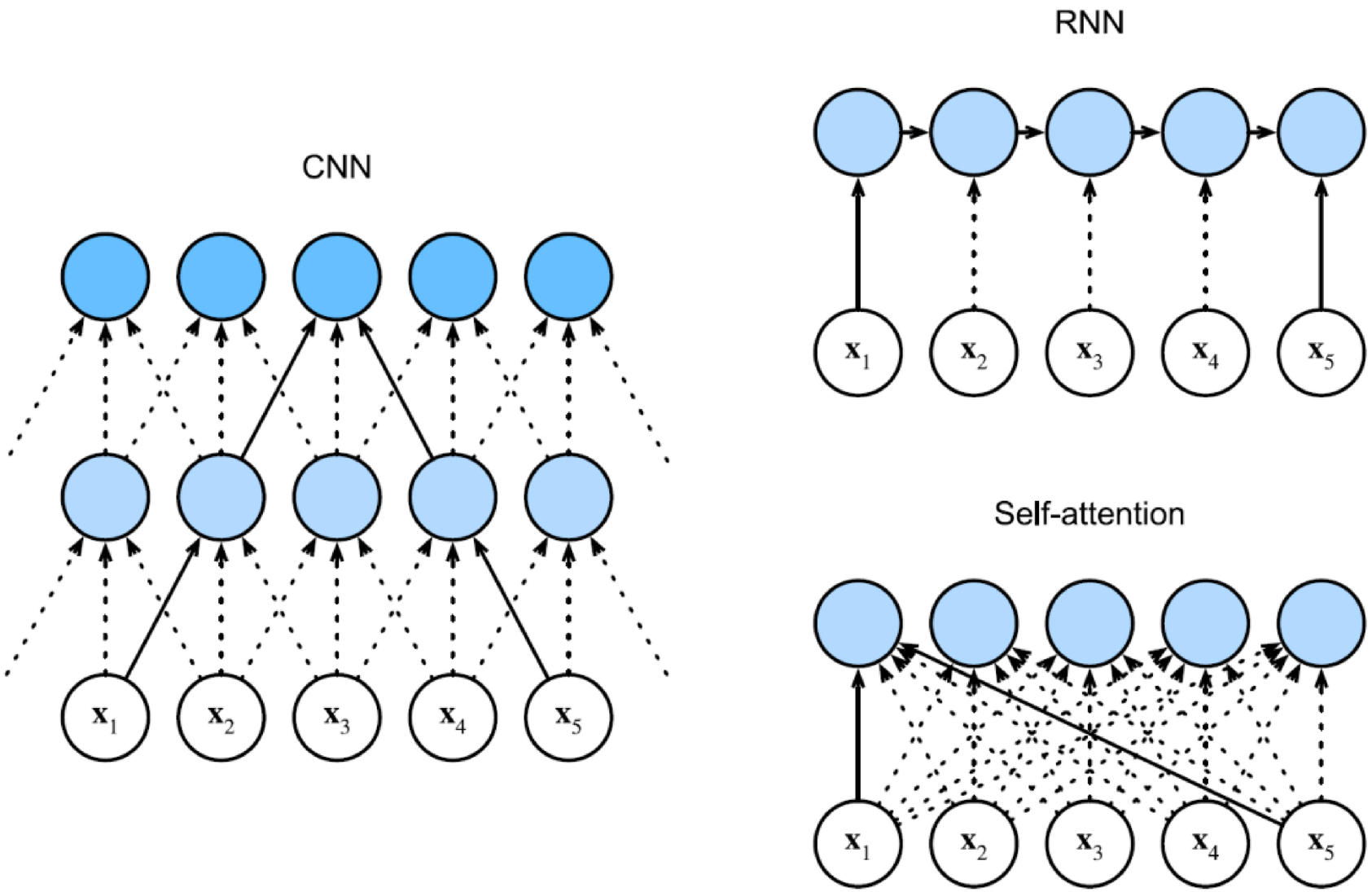


Figure 7: Comparing CNN (padding tokens are omitted), RNN, and self-attention architectures.

5.6 Self-attention and positional encoding

- consider a convolutional layer whose kernel size is k
- since the sequence length is n, the numbers of input and output channels are both d, the computational complexity of the convolutional layer is $\mathcal{O}(knd^2)$
- as Figure 7 shows, CNNs are hierarchical, so there are $\mathcal{O}(1)$ sequential operations and the maximum path length is $\mathcal{O}(n/k)$
- for example, $\boldsymbol{x}_1$ and $\boldsymbol{x}_5$ are within the receptive field of a two-layer CNN with kernel size 3 in Figure 7

5.6 Self-attention and positional encoding

- when updating the hidden state of RNNs, multiplication of the $d \times d$ weight matrix and the d-dimensional hidden state has a computational complexity of $\mathcal{O}(d^2)$
- since the sequence length is n, the computational complexity of the recurrent layer is $\mathcal{O}(nd^2)$
- according to Figure 7, there are $\mathcal{O}(n)$ sequential operations that cannot be parallelized and the maximum path length is also $\mathcal{O}(n)$

5.6 Self-attention and positional encoding

- in self-attention, the queries, keys, and values are all $n \times d$ matrices
- consider the scaled dot-product attention in (8), where a $n \times d$ matrix is multiplied by a $d \times n$ matrix, then the output $n \times n$ matrix is multiplied by a $n \times d$ matrix
- as a result, the self-attention has a $\mathcal{O}(n^2 d)$ computational complexity
- as we can see in Figure 7, each token is directly connected to any other token via self-attention
- therefore, computation can be parallel with $\mathcal{O}(1)$ sequential operations and the maximum path length is also $\mathcal{O}(1)$

5.6 Self-attention and positional encoding

- all in all, both CNNs and self-attention enjoy parallel computation, and self-attention has the shortest maximum path length
- however, the quadratic computational complexity with respect to the sequence length makes self-attention prohibitively slow for very long sequences
- unlike RNNs that recurrently process tokens of a sequence one by one, self-attention ditches sequential operations in favor of parallel computation
- to use the sequence order information, we can inject absolute or relative positional information by adding *positional encoding* to the input representations
- positional encodings can be either learned or fixed; in the following, we describe a fixed positional encoding based on sine and cosine functions

5.6 Self-attention and positional encoding

- suppose that the input representation $\boldsymbol{X} \in \mathbb{R}^{n \times d}$ contains the d-dimensional embeddings for n tokens of a sequence
- the positional encoding outputs $\boldsymbol{X} + \boldsymbol{P}$ using a positional embedding matrix $\boldsymbol{P} \in \mathbb{R}^{n \times d}$ of the same shape, whose element on the ith row and the $(2j)$th or the $(2j+1)$th column is:

$$
\begin{aligned}
p_{i,2j} &= \sin\left(\frac{i}{10000^{2j/d}}\right), \\
p_{i,2j+1} &= \cos\left(\frac{i}{10000^{2j/d}}\right).
\end{aligned} \tag{9}
$$

5.6 Self-attention and positional encoding

- in the positional embedding matrix $\boldsymbol{P}$, rows correspond to positions within a sequence and columns represent different positional encoding dimensions
- in the example in Figure 8, we can see that the 6th and the 7th columns of the positional embedding matrix have a higher frequency than the 8th and the 9th columns
- the offset between the 6th and the 7th (same for the 8th and the 9th) columns is due to the alternation of sine and cosine functions

5.6 Self-attention and positional encoding

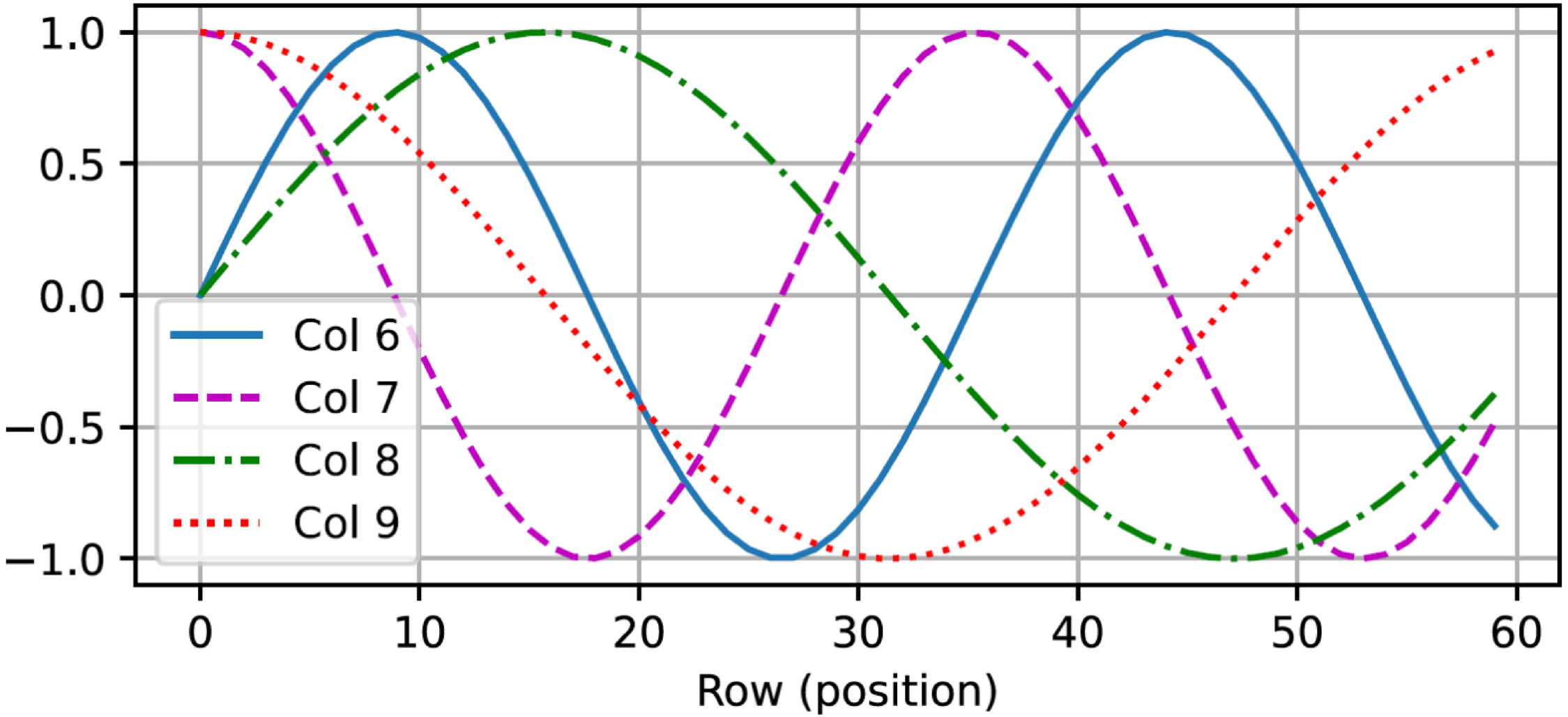


Figure 8: Positional encoding example.

5.6 Self-attention and positional encoding

- besides capturing absolute positional information, the above positional encoding also allows a model to easily learn to attend by relative positions
- this is because, for any fixed position offset δ, the positional encoding at position $i + \delta$ can be represented by a linear projection of that at position i
- this projection can be explained mathematically; denoting $\omega_j = 1/10000^{2j/d}$, any pair of $(p_{i,2j}, p_{i,2j+1})$ in (9) can be linearly projected to $(p_{i+\delta,2j}, p_{i+\delta,2j+1})$, for any fixed offset δ:

$$\begin{aligned}
\begin{bmatrix} \cos(\delta\omega_j) & \sin(\delta\omega_j) \\ -\sin(\delta\omega_j) & \cos(\delta\omega_j) \end{bmatrix} \begin{bmatrix} p_{i,2j} \\ p_{i,2j+1} \end{bmatrix} &= \begin{bmatrix} \cos(\delta\omega_j)\sin(i\omega_j) + \sin(\delta\omega_j)\cos(i\omega_j) \\ -\sin(\delta\omega_j)\sin(i\omega_j) + \cos(\delta\omega_j)\cos(i\omega_j) \end{bmatrix} \\
&= \begin{bmatrix} \sin((i+\delta)\omega_j) \\ \cos((i+\delta)\omega_j) \end{bmatrix} \\
&= \begin{bmatrix} p_{i+\delta,2j} \\ p_{i+\delta,2j+1} \end{bmatrix},
\end{aligned}$$

where the 2×2 projection matrix does not depend on any position index i